



Authors

timStorey@hexaponics.com

kentDelPino@hexaponics.com

At Hexaponics Ltd.

<https://twitter.com/hexaponics>

1

NarrowBand-Internet of Things (NB-IoT) in mixed city environment, the easy way.

NB-IoT is a Low Power Wide Area (LPWA) technology, depending on LTE/5G mobile tele infrastructure or private networks. It is meant for Massive machine type communication (mMTC). Tags: #UK5G #1nce #lorawan

The Purpose

At Hexaponics we have been testing in central London on our very small NB-IoT bridges for BLE v5 long range sensors, we wanted to find limitations of NB-IoT wireless technology with a hands-on approach. Our main concern is RF penetration down in basements, under concrete cover and in tunnels.

Our hypothesis being we would see more time-outs than actual data errors.

Hexaponics's distributed servers use the CoAP protocol which itself uses UDP/DTLS and we encode our payloads as CBOR or JSON.

Our NB-IoT bridges are also modems, where we meet the UDP layer. UDP itself discards erroneous packets and has no handshaking protocol i.e. errors may go unnoticed, we use CoAP's piggybacked ACK responses and their payload to give us visibility into this.

In addition to expected time-outs, we also expected to see variance in response times (ACK) with longer and greater variation under weak radio conditions as the underlying layers (LTE-NB) need to work harder to stay connected.

Image 1. Shows test locations.

We tested under different circumstances as shown, and in London tube stations!

We used these to get underground away from band-20 NB-IoT coverage.





The Result

At the mobile equipment, ME end (our NB-IoT bridge) we gathered signal and connection status using the modem command "AT+CSQ;+CREG?;+CGATT?\n". Sadly the modems do not support "+CESQ", for Extended Signal Quality read-out.

We did see the channel bit error rate (ber) go up, on several occasions when deep under-ground, we saw no timeouts during "normal" operation and we did not detect any data errors or payloads missing. Deep in the tube system we simply end up with disconnection, but we are warned about it via bit error rate (ber).

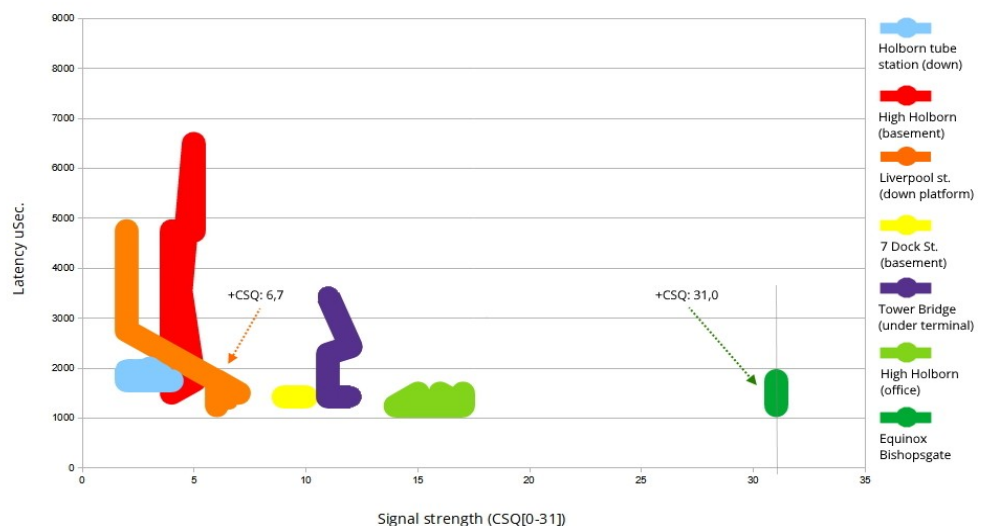
Since our interest was time variation measured both on the server side and out at the ME we used a schema as follows for our JSON payload (please note this is not a valid schema it is intended to explain the payload):

```
{
  "type": "object",
  "properties": {
    "f": {format, fixed to 6},
    "s": {source, "nb-iot test r00x", x[0..4]},
    "t": {date-time, from mobile network in Linux 32bit},
    "b": {counter, +1 each time our device tries to send},
    "x": {mobile.cell.signal, from +CSQ: XX <rsi>},
    "y": {UDP pay load(CoAP & JSON) sum-up counter in bytes},
    "z": {response time in millisec.; Average over 4 | 7 & last comm. set},
    "c": {Error down-counter[4..0] | [7..0]; note last comm. set}
  },
  "required": ["f", "s", "t", "b", "x", "y", "z", "c"]
}
```

The server receives these JSON objects as POST's in sets of 4 or 7 started at fixed time interval (28 sec.). The time between the 4 or 7 sets at the server side (POST arrival timestamp) vary due to the underlying radio conditions, as it was the case out the ME client, measured on the arrival of an ACK response, The ACK response time is stored in (z).

Graph 1. Shows how stable the response time (z; uSec) actually is, even under very different signal levels (x).

In environments with large moving machinery right next to ME (a sensor) there can be large variations in response times.





In Graph 1. the red measurements are taken in the basement behind 6 large lifts (12 people) and the orange measurements are taken in Liverpool st. tube station, on the westbound Central line platform while the train comes and leaves.

The Equipment

Hexaponics build our CoAP servers as sets of microservices written in Go and use AMPQ internally to maintain message queues, we use gRPC between services.

We try to avoid over reliance on third party libraries, but nor do we wish to reinvent any wheels. We use PostGRES for our relational tables and RabbitMQ for our message buffers. We have chosen not implement our own CoAP handling stack as it seemed more practical to use the recommended library <https://github.com/plgd-dev/go-coap> as referenced by the standards site itself <https://coap.technology/>.

We manage these deployments with Kubernetes and like to use modern CI techniques, i.e when we push feature code it is built, tested and then deployed into a test channel for sensor testing pre being rolled out as a production features.

The client API's that we are building are handled in a similar manner, i.e. Go, AMPQ, gRPC, PostGRES, K8s etc.

Hexaponics's bridges/gates are small devices with very low power consumption, based on System on Chip (SoC) supporting various wireless standards, including Matter, OpenThread and Zigbee for sensor communication. The cloud communication can be Swarm satellite based or NB-IoT/LTE-M for terrestrial based communications.

On the device, Hexaponics use proprietary and certified stacks from Silicon Labs and SIMCom Wireless Solutions etc. We tie the lower layer stacks together into a Real Time Operating System (RTOS) with our in house embedded CoAP and AT command interpreter stack.

For this test, 2 similar setups were used, based on Silicon Labs Thunderboard with ARMv8-M core and SIM7020E NB-IoT modules. The antennas used for the tests were of unknown origin, just labled "Multi Band PCB antenna" and were shipped with the NB-IoT modules. We have not tried to improve the antennas or NB-IoT modules performance. The SIM cards are from Ince and we were roaming on Vodafone UK net.

Image 2. Our test kits wrapped so it looks like an airline ticket, to make them London tube friendly.

The unknown antenna type can also be seen in the picture.

